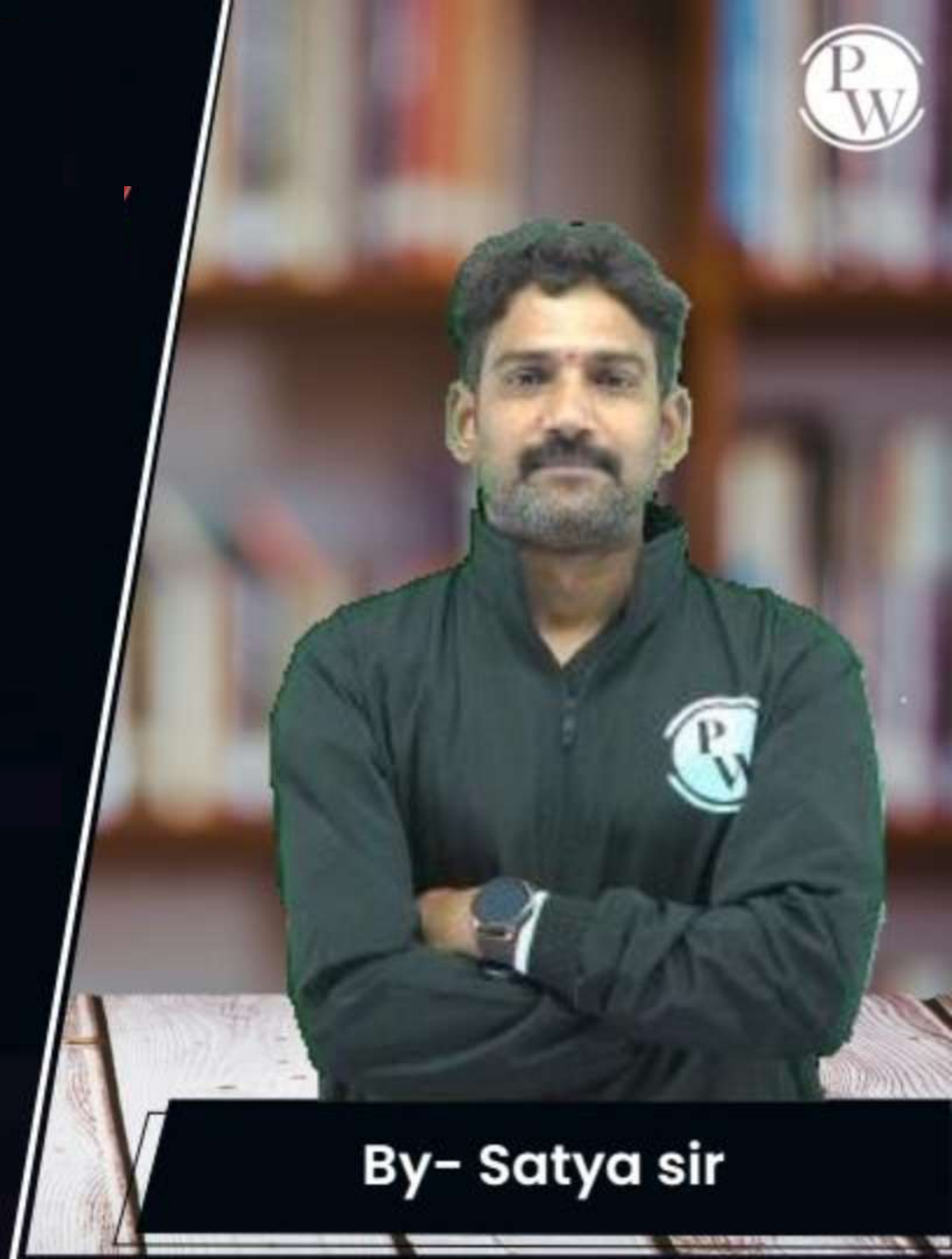
Data Science & AI

Data Structure through Python

Queues

DPP- 01

Discussion Notes



[MCQ]

#Q In queue, Deletion is happens at the_____end.

A. Rear

B. Front

C. Either rear or front

D. More than one of the above

E. None of the above

[MCQ]

#Q Carefully observe the below given queue (Simple Queue)

REAR	22	57	96	68	19	15	28	FRONT

How many dequeue operations needed to delete the largest element from the queue?

Largest Element

22	57	96	~~68~~	~~19~~	~~15~~	~~28~~

r

f (Deletion End)

dequeue( ) : 28
dequeue( ) : 15
dequeue( ) : 19
dequeue( ) : 68
dequeue( ) : 96

A 1

B 3

C 5 ✓

D More than one of the above

E None of the above

[MCQ]

#Q A queue is implemented using an array such that ENQUEUE and DEQUEUE operations are performed efficiently. Which one of the following statements is CORRECT (n refers to the number of items in the queue)?

O(1)
enqueue:
checkoverflow
else: rear=rear+1
Q[rear]=value
if f==-1; f=0

O(1)
dequeue:
check underflow
else: Value=Q[front]
front = front+1
if f>r : f=r=-1

A ✓ Both operations can be performed in O(1) time

B At most one operation can be performed in O(1) time but the worst case time for the other operation will be Ω(n)

C The worst-case time complexity for both operations will be Ω(n)

D Worst case time complexity for both operations will be Ω (log n)

[MCQ]

#Q A priority queue Q is used to implement a stack that stores characters. PUSH (C) is implemented INSERT (Q, C, K) where K is an appropriate integer key chosen by the implementation. POP is implemented as DELETEMIN(Q). For a sequence of operations, the keys chosen are in

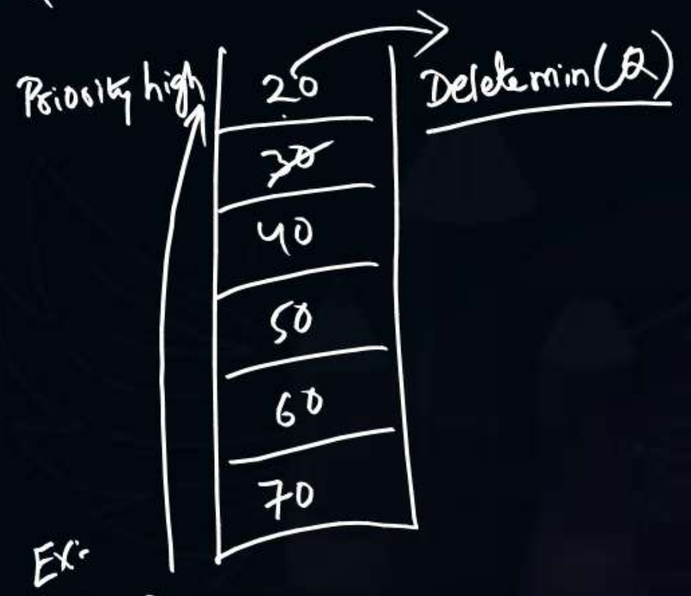
A non-increasing order

B non-decreasing order

C strictly increasing order

D strictly decreasing order ✓

Priority Queue delete Element with high Priority.



Strictly Decreasing Order.

[MCQ]

#Q The best data structure to check whether an arithmetic expression has balanced parentheses is a

A. Queue

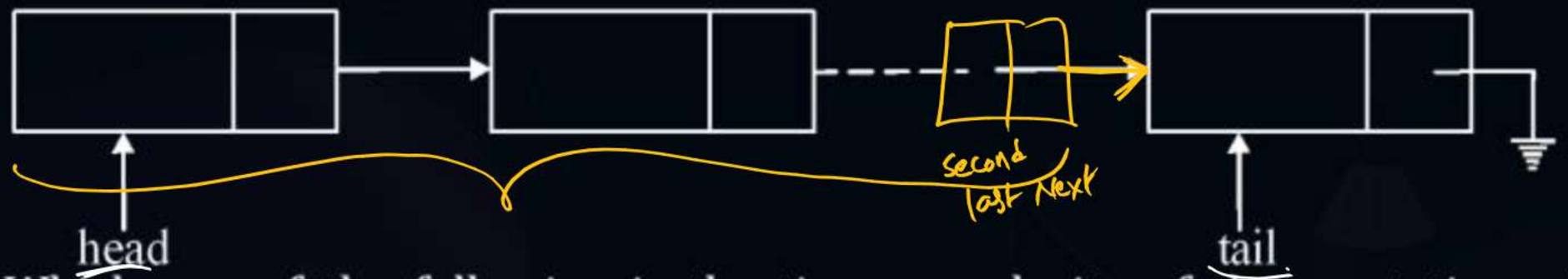
B. Stack ✓

C. Tree

D. List

[MCQ]

#Q A queue is implemented using a non-circular singly linked list. The queue has a head pointer and a tail pointer, as shown in the figure. Let n denote the number of nodes in the queue. Let (reference) enqueue be implemented by inserting a new node at the head, and dequeue be implemented by deletion of a node from the tail.



Which one of the following is the time complexity of the most time-efficient implementation of enqueue and dequeue, respectively, for this data structure?

A ✗ $\Theta(1), \Theta(1)$

B ✓ $\Theta(1), \Theta(n)$

C ✗ $\Theta(n), \Theta(1)$

D ✗ $\Theta(n), \Theta(n)$

Enqueue: (Insertion At Beginning)

```
New_Node = Node(data)
if self.head == None
    self.head = New_Node
else:
    New_Node.next = self.head, self.head = New_Node
```

$O(1)$

Dequeue(): Traverse from 1st Node Till Second last Node to change second last.next = None. Then, tail = None.

$O(n)$

THANK - YOU